Airline Reservation & Flight Operations Management System

Nhaithik Reddy Siddenki | 25WU0102185

B.Tech AI & ML | Faculty: Dr. C Nagaraju



MySQL • Python Flask • Triggers • 3NF

Problem and objectives

The problem



Seat conflicts

Two passengers booked into the same seat



Incomplete manifests

Records scattered across flights, tickets and check-ins



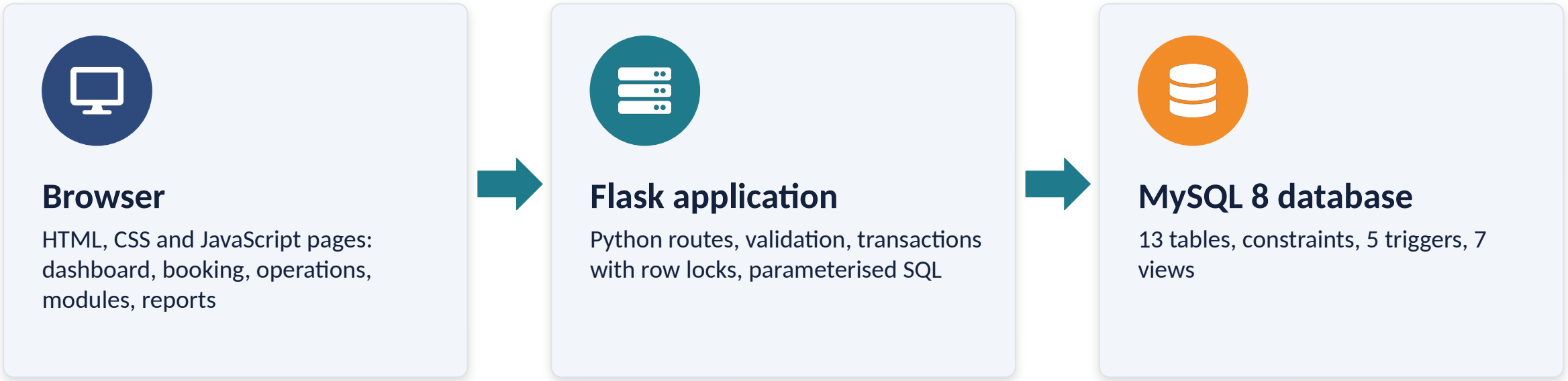
Invalid refunds

Cancellations after check-in or refunds above the amount paid

Objectives

1. Model airports, routes, aircraft, seats, flights, passengers, reservations and payments as an ER design
2. Normalise the schema to 3NF with keys, CHECK, DEFAULT and UNIQUE constraints
3. Enforce booking, check-in and refund rules with SQL triggers
4. Build a Python web app for search, booking, payment, ticketing, check-in, baggage, cancellation and reports

System architecture



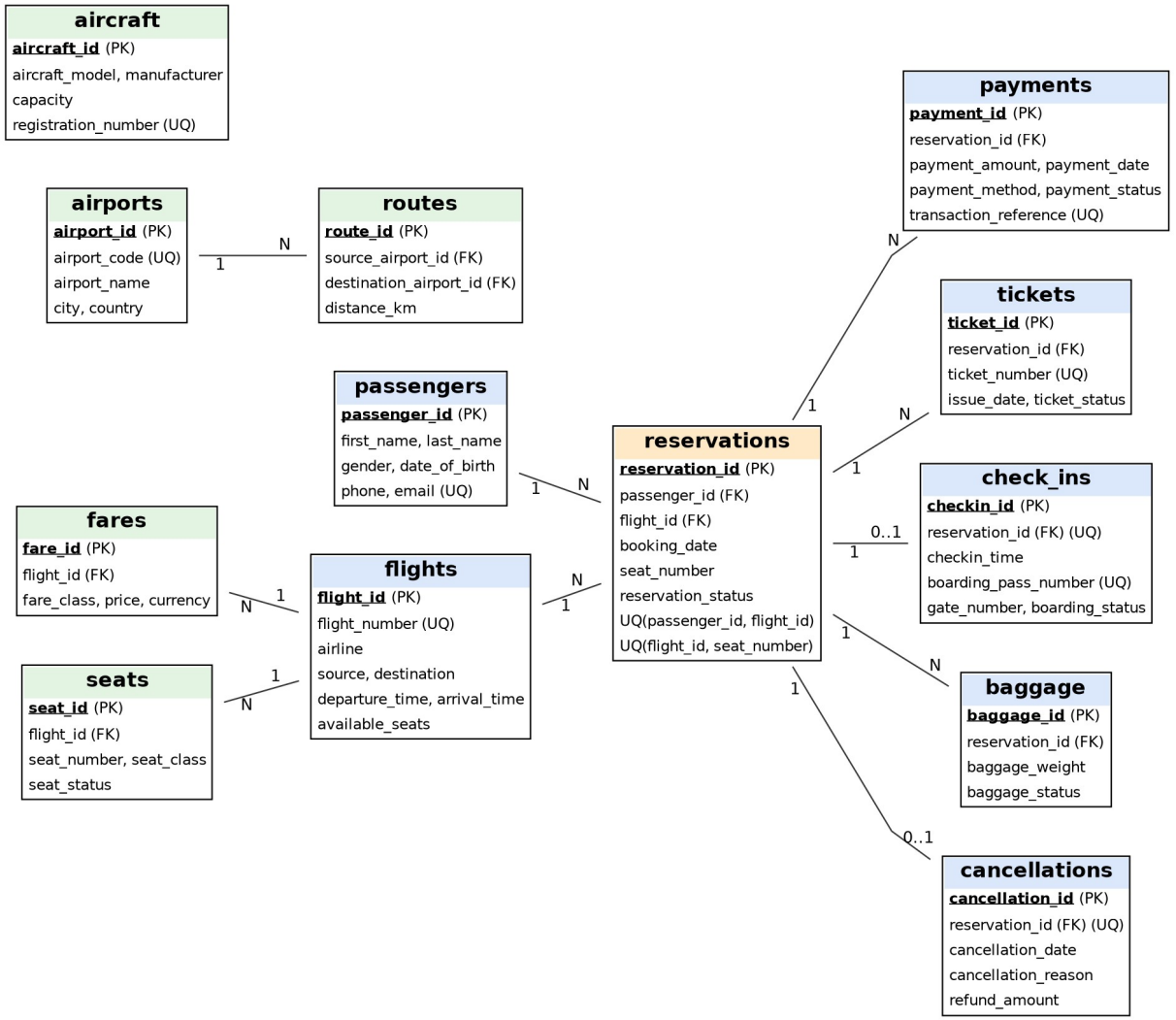
13
tables in 3NF

5
triggers

7
reporting views

8
workflow steps

ER diagram



Reservation is the hub
Links one passenger to one flight with a seat and a status

One-to-many
Passengers and flights to reservations; reservations to payments, tickets and baggage

One-to-one
A reservation has at most one check-in and one cancellation (UNIQUE)

Reference data
Airports, routes, aircraft, fares and seats (green)

Normalisation to 3NF

1NF

Atomic values

Every table has a primary key. Payments, tickets and baggage are separate rows, so there are no repeating groups.

2NF

No partial dependency

Single-column surrogate keys mean every attribute depends on the whole key.

3NF

No transitive dependency

Passenger and flight details live in their own tables and reservations only reference them.

Result: 13 relations in 3NF. Alternate keys (flight_number, email, ticket_number, boarding_pass_number, transaction_reference) are declared UNIQUE.

Design choice: flights stores source and destination as city names to keep search simple; airports and routes hold the reference data.

Integrity and business rules

Business rule	Enforced by
No two passengers in one seat	UNIQUE (flight_id, seat_number)
One reservation per passenger per flight	UNIQUE (passenger_id, flight_id)
Capacity limit	CHECK (available_seats >= 0); trigger blocks confirming at 0 seats
Non-negative baggage	CHECK (baggage_weight >= 0); app accepts 0 to 100 kg
Check-in only when Confirmed	Trigger before_checkin_insert
No cancellation after check-in	Trigger prevent_cancellation_after_checkin
Refund rules	Trigger before_cancellation_insert: refund up to the amount paid

Triggers keep seats consistent



Payment confirms: available_seats - 1 (blocked if no seats remain)

Cancellation: available_seats + 1

before_reservation_update and after_reservation_update (on reservations)

before_checkin_insert

Rejects check-in unless the reservation is Confirmed

prevent_cancellation_after_checkin

Rejects cancelling a reservation that has checked in

before_cancellation_insert

Rejects refunds on unpaid bookings or above the amount paid

Booking workflow in the app

1

Search

Filter flights by source, destination and date

2

Book

Pending reservation with passenger and seat

3

Seat

Click a free seat; change it until check-in

4

Pay

Valid payment confirms the reservation

5

Ticket

Issued only after payment

6

Check-in

Boarding pass and gate for Confirmed bookings

7

Baggage

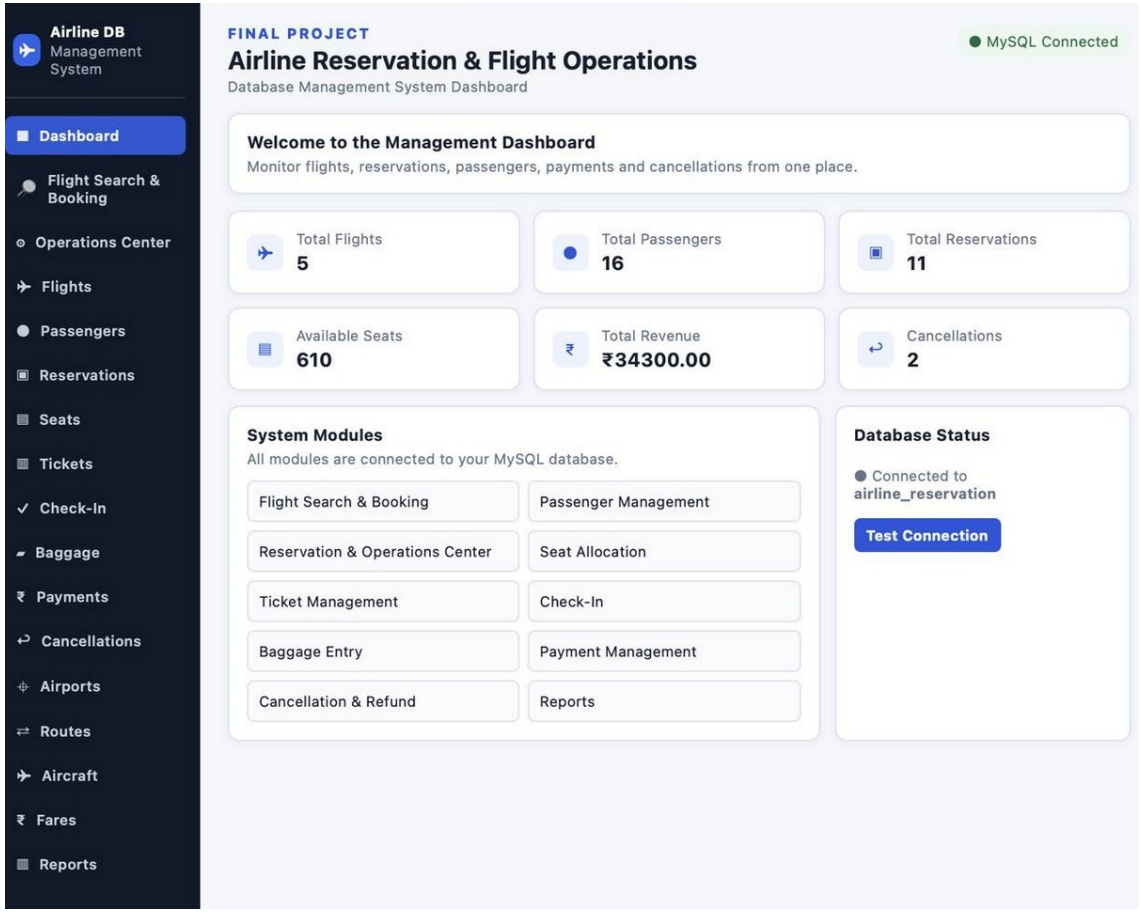
Weight recorded after check-in

8

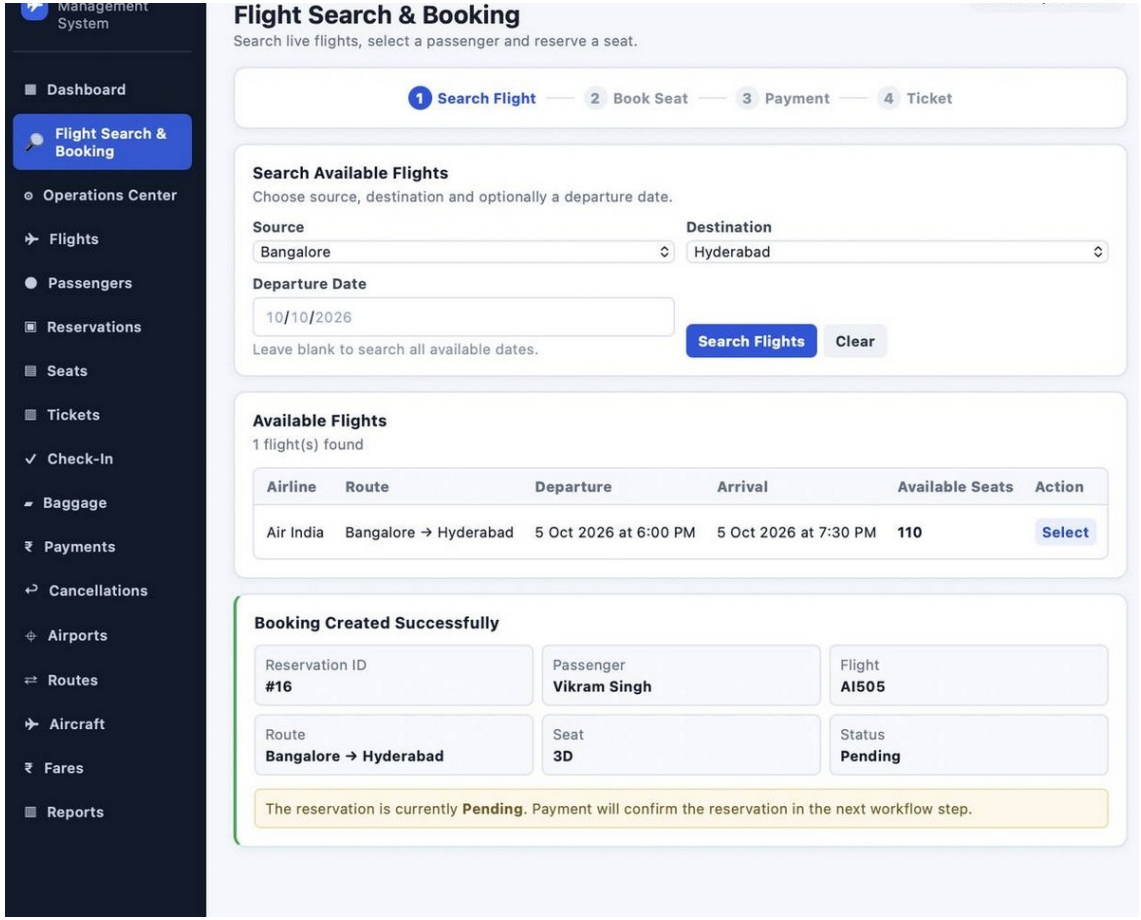
Cancel

Refund 90%, 50%, 25% or 0% by time to departure

Dashboard and flight booking

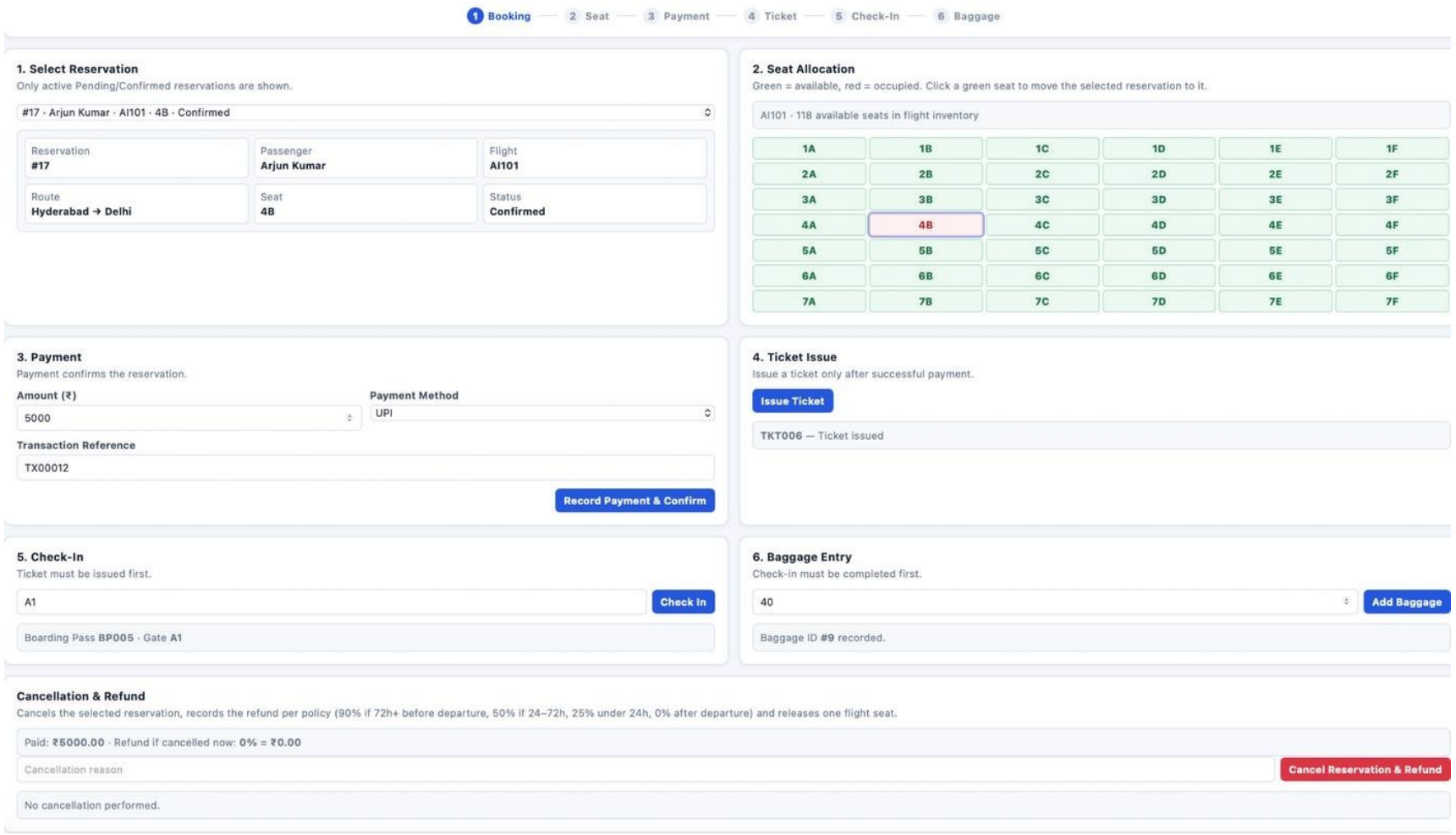


Live totals: 5 flights, 16 passengers, ₹34,300 revenue



Search Bangalore to Hyderabad; reservation #16 created as Pending

Operations Center



One reservation (#17, Arjun Kumar, flight AI101) taken through every step

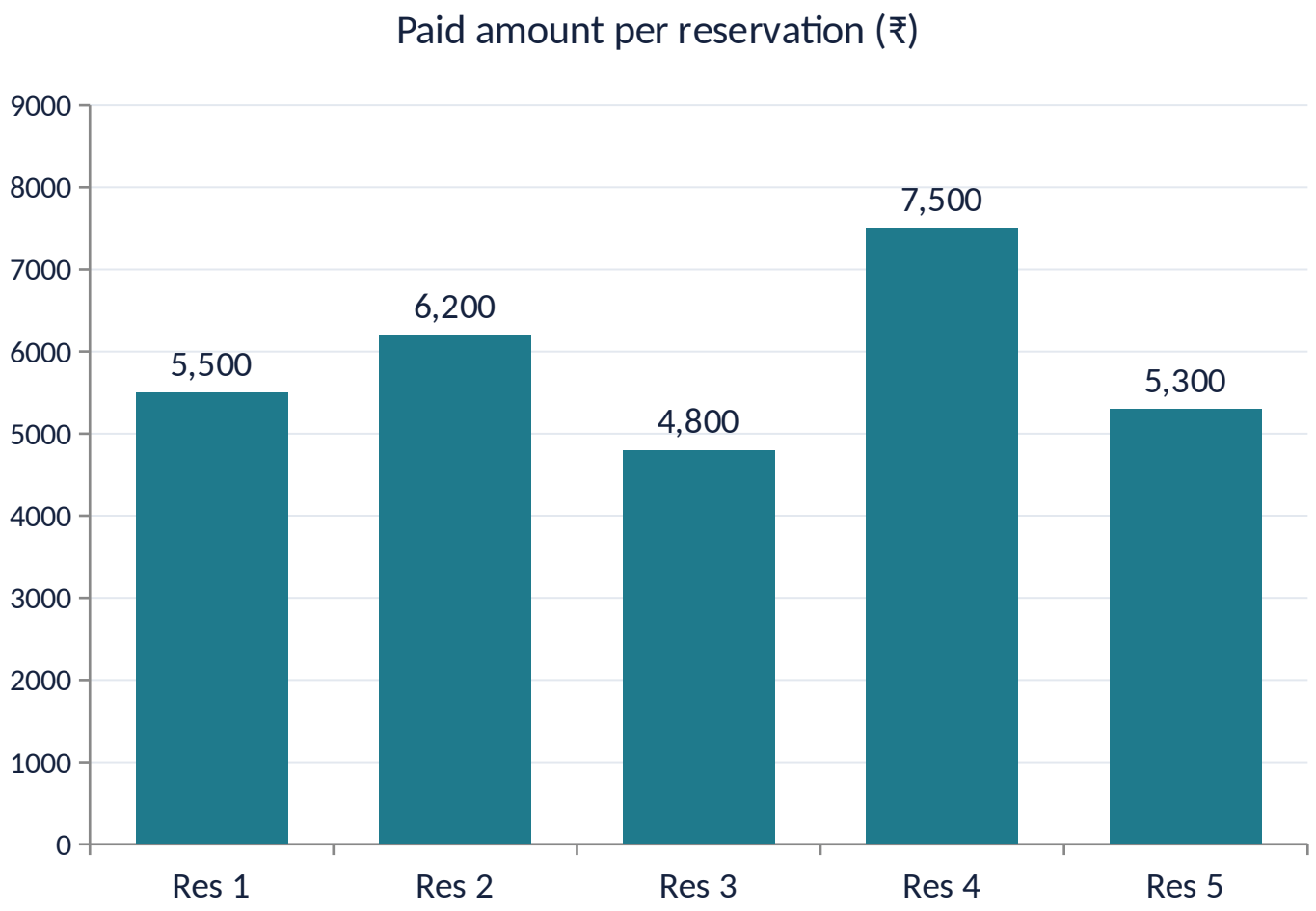
Payment
₹5,000 by UPI confirms #17

Ticket
TKT006 issued

Check-in
Boarding pass BP005, gate A1

Baggage
Record #9 saved, 40 kg

Reports from SQL views



₹29,300

total revenue from 5 paid payments

₹5,860

average payment

₹5,300

refunded across 2 cancellations

Seven views: manifest, occupancy, route demand, baggage, cancellations, revenue, reservation details

Testing: triggers block bad data

Test: Check-in on a Pending reservation

Result: Check-in is allowed only for Confirmed reservations

Test: Cancel after check-in

Result: Cannot cancel a reservation after check-in

Test: Refund ₹7,000 on a ₹6,200 payment

Result: Refund amount cannot exceed the payment amount

Test: Second cancellation of one reservation

Result: Duplicate entry for key cancellations.reservation_id

Positive path passed end to end: payment, ticket TKT006, boarding pass BP005 and baggage #9 for reservation #17. All four invalid actions were rejected with clear messages.

Outcomes and next steps

Achieved

- Normalised 13-table MySQL design with keys, CHECK and UNIQUE rules
- Five triggers enforcing check-in, cancellation, refund and seat rules
- Complete search-to-refund workflow in Python Flask
- Reports, sample data and tested rule violations

Next steps

- Login and user roles for staff and passengers
- Foreign keys linking flights to aircraft and routes
- Real payment gateway and automatic release of unpaid bookings
- Delay tracking for an on-time status report

Thank you

Questions?

Nhaithik Reddy Siddenki | 25WU0102185
github.com/Nhaithik/airline-reservation-flight-operations